
Project 2: Domain-Specific Named Entity Recognition (NER) for Danish

The Goal: Build a system that identifies specific entities (like names, organizations, or even banking terms) in Danish text, moving from a basic statistical baseline to a Transformer-based model.

1. What this implies (The Pipeline)

- **Preprocessing & Tokenization:**
 - As shown in your `tok_demo.py` and *Linguistics for NLP* slides, Danish has a "Deep Orthography" (complex relationship between spelling and sound). You would use the **Subword Segmentation** method described in the BERT slides (Slide 20, Seq II) to handle Danish's compound words (like *Motorcenter* or *EL-service*).
- **Data Labeling:**
 - You must implement the **BIO (Beginning, Inside, Outside) encoding** scheme explicitly mentioned in *Sequences I* (Slide 12).
 - *Example:* DANSKE (B-ORG) BANK (I-ORG).
- **The Baseline (The "Simple" Model):**
 - Per the *Experimental Setup* slides (Slide 42), you need a competitive baseline. You would implement a **Logistic Regression** classifier using hand-crafted features (Slide 9, *Intro to NN*) like "Is the word capitalized?" or "Does it have a specific suffix?".
- **The Deep Learning Model:**
 - **Architecture:** Use a **Bidirectional LSTM** (Slide 7, Seq II). This allows the model to see context to the left and right of a word, which is crucial for identifying entities.
 - **Training:** You would use **Backpropagation Through Time (BPTT)** (Slide 8, Seq I) and implement **Dropout** (Slide 17, Seq II) to prevent overfitting on your small Danish dataset.

2. Technical Requirements

- **HPC Usage:** Training the Bi-LSTM involves many parameters. You would use the `acctr` queue (GPU queue) on the ITU HPC as described in your *HPC Exercises* (Exercise 2).
- **Evaluation:** You must use **Span-F1** (Slide 13, Seq I) instead of simple accuracy, as NER is a "sparse" task (most words are 'O' tags).

Project 3: Controllable Text Generation (Style Transfer)

The Goal: Use a Generative Language Model to transform a sentence from one style to another (e.g., from "Internet Slang" to "Formal Danish/English") while preserving the meaning.

1. What this implies (The Pipeline)

- **Architecture (Encoder-Decoder):**
 - You would implement the **Encoder-Decoder model** shown in your *Sequences II* slides (Slide 11). The Encoder "crams" the source sentence into a vector, and the Decoder generates the new style.
 - To improve this, you would add the **Attention Mechanism** (Slide 13, Seq II) so the decoder can "attend" to specific source words during generation.
- **Advanced Model (Fine-tuning):**
 - Instead of training from scratch, you would take a **Pre-trained Decoder-only model** (like GPT-2) or a **Masked Language Model** (like BERT) and add a generation head (Slide 18, Seq II).
 - You would utilize the **Hugging Face library** (Module 13 in your course index) to load these models.
- **Hyperparameter Tuning:**
 - You would need to perform a **Grid Search** for the best Learning Rate and Batch Size, as illustrated in your *Sequences II* table (Slide 6).
 - You would implement a **Learning Rate Scheduler** with a "Warmup" phase (Slide 5, Seq II) to keep the training stable.

2. Technical Requirements

- **HPC Usage:** Generative models are VRAM-heavy. This project would imply using the **Nvidia DGX Spark machines** (Slide 15, *Intro to HPC*) or reserving a node with multiple GPUs via the `#SBATCH --gres=gpu:v100:4` command (Slide 18, *Intro to HPC*).
- **Evaluation:**
 - **Intrinsic:** Use **Perplexity** (Slide 2, Seq II) to see how "fluent" the generated text is.
 - **Extrinsic:** Use **BLEU scores** (Slide 14, Seq II) to measure how close the output is to a human-written formal version.

Key Comparison for your Report

Feature	Project 2 (NER)	Project 3 (Style Transfer)
Linguistic Level	Morphology & Syntax	Semantics & Pragmatics
Key Algorithm	Bi-LSTM (RNN)	Transformers (Attention)
HPC Need	Moderate (Single GPU)	High (Multi-GPU/DGX Spark)
Baseline	Logistic Regression	Pre-trained Language Model

Recommendation: If you want to dive deep into the **Linguistics for NLP** aspect of the course, choose **Project 2**. It forces you to deal with the "Deep Orthography" of Danish. If you are more interested in **Engineering and Scaling**, choose **Project 3**, as it focuses on the "Transformer Revolution" and using the high-end DGX Spark hardware.

<https://arxiv.org/html/2402.18209v1#S5>

<https://d-nb.info/1313510920/34>

NLP Project Notes: Danish Named Entity Recognition

Comprehensive Guide to Theory, Methods, Datasets and Project Ideas

Prepared for NLP and Deep Learning Course Project

=====

PROFESSOR CHRISTIAN'S GUIDANCE AND RECOMMENDATIONS

=====

Key Recommendations from Professor's Email:

1. **BASELINE MODEL:** Use BiLSTM or BERT (NOT logistic regression)
 - Reason: NER is a sequence labelling task
 - Logistic regression at token level does NOT model sequence structure
 - Will perform poorly because it ignores dependencies between labels
2. **MAKE IT INTERESTING:** Add advanced comparisons beyond single baseline
 - Options suggested:
 - a) Transfer learning with different datasets or languages
 - b) Data augmentation techniques
 - c) Ensemble models
 - d) Systematic comparison of different BERT models
3. **AMBITION LEVEL:** Be more ambitious than basic binary comparison
 - Compare multiple approaches
 - Show systematic evaluation
 - Demonstrate understanding of sequence modeling

=====

PART 1: CORE NER THEORY AND FUNDAMENTAL CONCEPTS

=====

1.1 What is Named Entity Recognition (NER)?

Definition:

NER is a sequence labelling task in Natural Language Processing where each token (word or subword) in a sentence receives a label indicating whether it is part of a named entity and what type.

Entity Types (Common in CoNLL/DaNE datasets):

- **PERSON (PER):** Individual names
 - Examples: "Christian Hansen", "Queen Margrethe", "Messi"
- **ORGANIZATION (ORG):** Companies, institutions, government bodies
 - Examples: "Danske Bank", "ITU", "European Union", "Microsoft"
- **LOCATION (LOC):** Geographic locations (cities, countries, natural features)
 - Examples: "Copenhagen", "Denmark", "Roskilde Fjord", "Zealand"
- **MISCELLANEOUS (MISC):** Everything else (nationalities, events, products, dates)
 - Examples: "Danish", "Olympics", "iPhone", "Nobel Prize"

Why NER Matters:

- Information Extraction: Extract structured data from unstructured text
- Knowledge Graphs: Identify entities as nodes in knowledge bases
- Question Answering: Find answers to "Who?", "Where?", "Which company?"
- Document Classification: Use entities as features
- Financial Analysis: Track mentions of companies, locations in news
- Medical Records: Extract drug names, diseases, procedures

1.2 BIO Encoding Scheme - CRITICAL CONCEPT

Definition:

BIO (also called IOB) is the standard encoding scheme for sequence labelling. It handles multi-token entities by marking the Beginning, Inside, and Outside of entity spans.

BIO Tags:

- B-X: Beginning of entity type X
- I-X: Inside (continuation) of entity type X
- O: Outside any entity (not part of a named entity)

Detailed Example:

Sentence: "Danske Bank opened a branch in Copenhagen yesterday."

Tokenization and BIO tags:

Token	BIO Tag	Description
Danske	B-ORG	(beginning of organization)
Bank	I-ORG	(inside/continuation of organization)
opened	O	(not an entity)
a	O	
branch	O	
in	O	
Copenhagen	B-LOC	(beginning and only token of location)
yesterday	O	
.	O	

Why BIO and not just entity types?

- Problem: Multi-token entities need boundaries
- Example: "Bank of America" vs "American Bank" - both have "Bank" and "America/n", but different structures
- BIO makes it clear where entities start and end
- Prevents ambiguity in consecutive entities: "Apple CEO Tim Cook" = B-ORG I-ORG B-PER I-PER

Why This Matters for Models:

- Models must learn valid BIO transitions (e.g., I-PER cannot follow B-ORG)
- Token-level classifiers (logistic regression) cannot enforce these constraints
- Sequence models (BiLSTM, BERT + CRF) can learn and enforce valid transitions

1.3 Sequence Labelling vs Token Classification

Token Classification (Why Logistic Regression Fails):

- Treats each token independently
- No context from neighboring tokens

- Cannot enforce valid label sequences
- Example problem: might predict I-PER after O (invalid transition)
- No memory of previous predictions

Sequence Labelling (BiLSTM, BERT approach):

- Models dependencies between labels
- Uses context from entire sentence
- Can learn that "Bank" after "Danske" is likely I-ORG not B-ORG
- Enforces valid transitions (especially with CRF layer)

Analogy:

- Token classification = predicting each word of a sentence independently
- Sequence labelling = understanding the sentence as a whole

This is WHY the professor said logistic regression "is unlikely to perform very well"!

1.4 Danish Language Challenges for NER

Compound Words:

Danish frequently forms long compound words without spaces:

- "Motorcenter" = motor + center
- "EL-service" = electrical + service
- "Sundhedsstyrelsen" = health + authority

Problem: These are often out-of-vocabulary (OOV) for word-based models

Solution: Subword tokenization (BPE, WordPiece used by BERT)

Deep Orthography:

- Less consistent spelling-to-pronunciation rules than English
- More morphological variation
- Character-level features can help

Case Sensitivity:

- Like English, capitalization helps identify entities
- But: sentence-initial words are always capitalized
- Danish nouns can be capitalized (older texts)

Why This Matters:

- Subword tokenization is almost mandatory for Danish
- BERT's WordPiece handles this naturally
- Pure word-level models will struggle with compounds

=====

PART 2: MODEL ARCHITECTURES - DETAILED EXPLANATIONS

=====

2.1 BiLSTM (Bidirectional Long Short-Term Memory) - BASELINE

What is LSTM?

- A type of Recurrent Neural Network (RNN)
- Designed to handle sequential data
- Has "memory cells" that can remember information over long sequences

- Solves the vanishing gradient problem of vanilla RNNs

Why Bidirectional?

- Reads sentence left-to-right AND right-to-left
- Combines both contexts for each token
- Example: "bank" in "river bank" vs "Danske Bank"
 - Forward LSTM sees "river" before "bank"
 - Backward LSTM sees context after "bank"
 - Combined = better disambiguation

BiLSTM Architecture for NER:

Input Layer:

- Word embeddings (e.g., 300-dim pre-trained or trained from scratch)
- Optional: character-level CNN/LSTM to handle OOV words
- Optional: features (capitalization, POS tags)

BiLSTM Layers:

- Forward LSTM: processes tokens left-to-right
- Backward LSTM: processes tokens right-to-left
- Concatenate both hidden states
- Typical hidden size: 128-256 per direction
- Can stack 1-2 BiLSTM layers
- Dropout between layers (0.3-0.5) to prevent overfitting

Output Layer:

- Dense layer mapping BiLSTM output to number of BIO tags
- Softmax for probabilities
- Optional: CRF layer for valid transition enforcement

Advantages:

- Explicitly models sequence structure (unlike logistic regression)
- Can capture long-range dependencies
- Relatively interpretable
- Works with smaller datasets than BERT

Disadvantages:

- Slower than feedforward networks
- Harder to parallelize than BERT
- Needs careful tuning (hidden size, dropout, layers)
- Performance typically lower than BERT on sufficient data

Typical Performance on DaNE-style datasets:

- BiLSTM alone: ~82-85% span-F1
- BiLSTM + CRF: ~85-87% span-F1
- BiLSTM + char embeddings + CRF: ~87-89% span-F1

2.2 BERT for NER - ADVANCED BASELINE

What is BERT?

- Bidirectional Encoder Representations from Transformers
- Pre-trained on massive text corpora (billions of words)
- Uses self-attention mechanism instead of recurrence

- Produces contextual embeddings (same word gets different embedding based on context)

Key Concepts:

Transformer Architecture:

- Multi-head self-attention: each token attends to all other tokens
- Fully parallelizable (faster than LSTM)
- Multiple layers (12 for BERT-base, 24 for BERT-large)
- Each layer refines representations

Pre-training Tasks:

- Masked Language Modeling (MLM): predict masked words from context
- Next Sentence Prediction (NSP): predict if two sentences are consecutive
- This pre-training gives BERT general language understanding

Subword Tokenization (WordPiece):

- Vocabulary of ~30,000 subword units
- Handles Danish compounds naturally:
 - "Sundhedsstyrelsen" → ["Sund", "##heds", "##styrelsen"]
- Rare words split into known pieces
- ## prefix indicates continuation of previous token

BERT for NER - Fine-tuning Approach:

1. Start with pre-trained BERT weights
2. Add classification head on top:
 - Linear layer: hidden_size (768 or 1024) → num_labels (9 for DaNE)
 - Optional: Add CRF layer for transition constraints
3. Fine-tune entire model on NER data:
 - Very small learning rate (1e-5 to 5e-5)
 - Few epochs (2-5 typically enough)
 - Batch size 8-32 (limited by GPU memory)
4. Handle subword alignment:
 - Problem: "Copenhagen" becomes ["Co", "##pen", "##hagen"]
 - Solutions:
 - a) Only predict on first subword, ignore ## tokens
 - b) Predict on all subwords, take first prediction
 - c) Average predictions across subwords

BERT Variants for Danish NER:

1. Danish BERT (botXO/DaBERT)
 - Pre-trained specifically on Danish text
 - Best for monolingual Danish NER
 - Smaller pre-training corpus than mBERT
2. Multilingual BERT (mBERT)
 - Pre-trained on 104 languages including Danish
 - Good for cross-lingual transfer
 - Base and Large versions available
3. XLM-RoBERTa (XLM-R)
 - Trained on 100 languages
 - Often outperforms mBERT

- Better cross-lingual capabilities
- Base and Large versions

4. Nordic BERT

- Pre-trained on Nordic languages
- Good middle ground for Danish

Advantages of BERT for NER:

- State-of-the-art performance (often 3-5% F1 over BiLSTM)
- Handles subwords naturally (great for Danish compounds)
- Rich contextual representations
- Transfer learning from massive pre-training
- Parallel processing (faster inference than BiLSTM)

Disadvantages:

- Requires more GPU memory
- Needs careful hyperparameter tuning
- Can overfit on very small datasets
- Less interpretable than BiLSTM
- Larger model size

Typical Performance on DaNE:

- mBERT base: ~89-91% span-F1
- Danish BERT: ~90-92% span-F1
- XLM-R large: ~91-93% span-F1

2.3 Additional Important Concepts

Conditional Random Field (CRF) Layer:

What is it?

- A structured prediction layer added on top of BiLSTM or BERT
- Models transitions between BIO tags
- Uses Viterbi algorithm for decoding

Why use CRF?

- Enforces valid tag sequences:
 - Invalid: O $\rightarrow$ I-PER (cannot start entity with I-tag)
 - Invalid: B-ORG $\rightarrow$ I-PER (cannot change entity type mid-span)
 - Valid: B-ORG $\rightarrow$ I-ORG $\rightarrow$ O
- Learns transition probabilities:
 - $P(\text{I-PER} \mid \text{B-PER})$ should be high
 - $P(\text{I-LOC} \mid \text{B-PER})$ should be zero
- Typically adds 1-2% F1 improvement

When to use:

- Almost always beneficial for BiLSTM
- Less critical for BERT (already captures dependencies well)
- Adds computational cost during training and inference

Transfer Learning:

Definition:

- Using knowledge from one task/language/domain to improve another
- Central to modern NLP with pre-trained models

Types relevant to NER project:

1. Cross-lingual Transfer:

- Train on English CoNLL 2003
- Evaluate/fine-tune on Danish DaNE
- Tests if entity recognition transfers across languages
- mBERT and XLM-R are designed for this

2. Domain Transfer:

- Train on news domain (DaNE)
- Evaluate on financial/legal domain
- Tests domain adaptation

3. Few-shot Transfer:

- Pre-train on large English data
- Fine-tune on small Danish subset (10%, 25%, 50%)
- Shows sample efficiency

Why This is Interesting (per professor's guidance):

- Goes beyond simple model comparison
- Shows understanding of modern NLP techniques
- Practical value for low-resource languages like Danish
- Can compare multiple transfer strategies

Dropout and Regularization:

Dropout:

- Randomly zeroes activations during training
- Prevents co-adaptation of neurons
- Critical for LSTM (use 0.3-0.5)
- Less critical for BERT (already has regularization from pre-training)

Other Regularization:

- Weight decay: L2 penalty on parameters
- Early stopping: stop when validation loss stops improving
- Gradient clipping: prevent exploding gradients in LSTM

PART 3: EVALUATION METRICS

3.1 Why NOT Token-level Accuracy?

Problem with accuracy:

- Most tokens are O (outside entities)
- Example: "The Danske Bank CEO visited Copenhagen yesterday"
- Only 4/7 tokens are entities
- Model predicting all O gets 43% tokens correct!
- But completely fails at NER

- Class imbalance makes accuracy meaningless

3.2 Span-F1 Score (PRIMARY METRIC)

What is a span?

- A complete, correctly-bounded entity with correct type
- "Danske Bank" tagged as B-ORG I-ORG = one ORG span

Span-F1 Calculation:

Precision = (Correct predicted spans) / (All predicted spans)

Recall = (Correct predicted spans) / (All gold spans)

F1 = $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$

Example:

Gold: [Danske Bank]_ORG opened in [Copenhagen]_LOC

Prediction: [Danske]_ORG [Bank]_ORG opened in [Copenhagen]_LOC

- Gold spans: {"Danske Bank"/ORG, "Copenhagen"/LOC} = 2
- Predicted spans: {"Danske"/ORG, "Bank"/ORG, "Copenhagen"/LOC} = 3
- Correct: {"Copenhagen"/LOC} = 1

Precision = $1/3 = 33.3\%$

Recall = $1/2 = 50.0\%$

F1 = $2 * (0.333 * 0.5) / (0.333 + 0.5) = 40.0\%$

Why span-F1?

- Measures actual NER performance (finding complete entities)
- Boundary errors are penalized (splitting "Danske Bank" hurts F1)
- Type errors are penalized (tagging ORG as PER counts as 0)
- Standard metric in NER literature (CoNLL 2003, DaNE papers)

3.3 Per-entity-type F1

- Calculate F1 separately for PER, ORG, LOC, MISC
- Shows which entity types are hard
- Typical pattern:
 - PER and LOC: easier (higher F1)
 - ORG: harder (ambiguous, variable length)
 - MISC: hardest (catch-all category)

3.4 Macro vs Micro Averaging

Macro-F1:

- Average F1 across entity types
- Treats all types equally
- Good when types are balanced

Micro-F1:

- Pool all predictions, calculate one F1
- Dominated by frequent entity types
- Typical choice for NER (what we usually report)

PART 4: DATASETS

4.1 DaNE (Danish Named Entities) - PRIMARY DATASET

Overview:

- Danish NER dataset based on Universal Dependencies
- News domain text from Danish newspapers
- BIO-tagged with 4 entity types: PER, ORG, LOC, MISC

Statistics:

- Approximately 5,600 sentences
- Approximately 100,000 tokens
- Split into train/dev/test (typically 80/10/10 or similar)

Entity distribution (approximate):

- PER (Person): ~30%
- ORG (Organization): ~25%
- LOC (Location): ~30%
- MISC (Miscellaneous): ~15%

Format:

- CoNLL-style format (one token per line, blank line = sentence boundary)
- Columns: token, POS tag, syntactic label, NER tag

Example entry:

```
Danske  PROPN  _   B-ORG
Bank    PROPN  _   I-ORG
har     VERB   _   O
kontor  NOUN   _   O
i       ADP    _   O
Copenhagen PROPN _   B-LOC
.
```

Challenges:

- Moderate size (smaller than English CoNLL 2003)
- Domain-specific (news, may not generalize to other domains)
- Danish compound words
- Nested entities sometimes ambiguous

Why use DaNE:

- Standard benchmark for Danish NER
- Comparable to CoNLL 2003 format
- Allows comparison with published results
- Good size for course project (not too large, not too small)

4.2 CoNLL 2003 - TRANSFER LEARNING SOURCE

Overview:

- English NER dataset (can also use German, Dutch versions)
- Reuters news articles

- Same 4 entity types: PER, ORG, LOC, MISC
- BIO encoding

Statistics:

- Training: ~14,000 sentences, ~200,000 tokens
- Dev: ~3,000 sentences
- Test: ~3,000 sentences
- Much larger than DaNE!

Why use CoNLL for transfer:

- Same task (4-class NER)
- Same format (BIO encoding)
- Much more training data
- Test cross-lingual transfer with mBERT/XLM-R
- Standard baseline in literature

Transfer learning scenarios:

1. Zero-shot: Train on English, test directly on Danish
2. Few-shot: Train on English, fine-tune on small Danish subset
3. Full fine-tune: Train on English, fine-tune on full Danish

4.3 Data Preprocessing for Danish NER

Tokenization:

- Use subword tokenizers (BERT WordPiece, BPE)
- Handle Danish special characters: æ, ø, å
- Preserve case (important for NER)

Handling compound words:

- BERT's WordPiece automatically splits:
"Sundhedsstyrelsen" → ["Sund", "##heds", "##styrelsen"]
- Align BIO tags to first subword token
- Example:
Token: Sundhedsstyrelsen B-ORG
Subwords: [Sund (B-ORG), ##heds (X), ##styrelsen (X)]
Where X = ignore in loss calculation

Data splits:

- Standard: 80% train / 10% dev / 10% test
- OR use official DaNE splits if provided
- Stratify by document to avoid sentence leakage

Data augmentation (optional for project):

1. Entity replacement:
 - Replace "Copenhagen" with other Danish cities
 - Replace "Danske Bank" with other banks
 - Preserves sentence structure
2. Back-translation:
 - Translate Danish → English → Danish
 - Creates paraphrases
 - May introduce errors

3. Synonym replacement:
- Replace non-entity words with synonyms
 - Increases vocabulary diversity

=====

PART 5: PROJECT IDEAS (ALIGNED WITH PROFESSOR'S GUIDANCE)

=====

REMINDER: Professor wants BiLSTM or BERT as baseline + interesting comparison
Options: transfer learning, data augmentation, ensembles, or BERT variant comparison

PROJECT IDEA 1: Cross-Lingual Transfer Learning Study

Goal: Evaluate how transfer learning from English improves Danish NER

Baseline Models:

- BiLSTM trained only on DaNE (monolingual baseline)
- mBERT fine-tuned only on DaNE (monolingual BERT baseline)

Transfer Learning Experiments:

1. Zero-shot transfer:

- Train mBERT on English CoNLL 2003
- Evaluate directly on Danish DaNE (no Danish training)
- Shows cross-lingual capability

2. Few-shot scenarios:

- Pre-train on English CoNLL
- Fine-tune on 10%, 25%, 50% of DaNE training data
- Compare to training only on those small DaNE subsets
- Shows sample efficiency gains

3. Full transfer:

- Pre-train on English
- Fine-tune on full DaNE
- Compare to DaNE-only training

Models to Compare:

- mBERT (multilingual, designed for this)
- XLM-RoBERTa (better cross-lingual)
- Danish BERT (monolingual, no cross-lingual capability - control)

Expected Results:

- Zero-shot: ~60-70% F1 (shows language transfer works)
- Few-shot with 10% DaNE: transfer beats no-transfer by 5-10% F1
- Full transfer: marginal gains (1-2% F1) over DaNE-only

Why Interesting:

- Demonstrates modern transfer learning
- Practical for low-resource scenarios
- Systematic comparison across data sizes
- Can plot learning curves showing transfer benefits

PROJECT IDEA 2: Systematic BERT Variant Comparison

Goal: Compare different BERT models for Danish NER systematically

Baseline:

- BiLSTM + CRF on DaNE (non-BERT baseline)

BERT Models to Compare:

1. Danish BERT (Certainly/BotXO Danish BERT)
 - Monolingual Danish pre-training
 - Smaller pre-training corpus
2. mBERT base
 - 104 languages including Danish
 - Standard multilingual model
3. mBERT large (if GPU memory allows)
 - Same as base but larger (24 layers vs 12)
 - More parameters = better performance?
4. XLM-RoBERTa base
 - 100 languages
 - RoBERTa training (often beats BERT)
5. XLM-RoBERTa large
 - Largest model, best expected performance
6. Nordic BERT (if available)
 - Specialized for Nordic languages
 - Middle ground between Danish-only and multilingual

Evaluation Protocol:

- Same preprocessing for all
- Same hyperparameters where possible (learning rate, batch size)
- Same train/dev/test splits
- Report span-F1 and per-entity-type F1
- Measure training time and inference speed
- Analyze errors: which entities does each model struggle with?

Additional Comparison Dimensions:

- Model size (parameters)
- Training time
- Inference speed
- Performance vs efficiency trade-off

Expected Results:

- BiLSTM: ~87% F1
- Danish BERT: ~90% F1
- mBERT base: ~89% F1
- XLM-R base: ~90% F1
- XLM-R large: ~92% F1 (best, but slowest)

Why Interesting:

- Systematic model comparison (professor's suggestion)

- Shows trade-offs (performance vs speed vs size)
- Practical guidance for choosing models
- Can extend with CRF on/off comparison

PROJECT IDEA 3: Data Augmentation Impact Study

Goal: Quantify impact of data augmentation on low-resource Danish NER

Baseline:

- BiLSTM trained on full DaNE
- BERT trained on full DaNE

Low-Resource Scenarios:

Create subsets: 10%, 25%, 50%, 100% of DaNE training data

Augmentation Techniques:

1. Entity Swapping:

- Build gazetteer of Danish person names, companies, cities
- Replace entities in sentences with others of same type
- Example: "Lars besøgte Aalborg" → "Mette besøgte Odense"
- Preserves sentence structure and context

2. Contextual Word Replacement:

- Use Danish word embeddings to find synonyms
- Replace non-entity words
- Keep entities unchanged

3. Back-Translation:

- Translate Da → En → Da using translation API
- Creates natural paraphrases
- Re-align entity labels

4. Mixup or other modern techniques

Experimental Design:

For each data size (10%, 25%, 50%, 100%):

- Train baseline (no augmentation)
- Train with entity swapping (2x data)
- Train with back-translation (2x data)
- Train with combined augmentation (3x data)

Evaluation:

- Span-F1 on held-out test set
- Learning curves showing data efficiency
- Statistical significance testing

Expected Results:

- Biggest gains at 10% and 25% (low resource)
- Diminishing returns at 100% (sufficient data)
- Entity swapping most reliable
- Back-translation can introduce noise

Why Interesting:

- Data augmentation is professor's suggestion
- Practical value for low-resource settings
- Can combine with transfer learning
- Shows when augmentation helps vs hurts

PROJECT IDEA 4: Ensemble Methods for Danish NER

Goal: Test if ensembles outperform single models

Base Models:

1. BiLSTM + CRF (diverse architecture)
 2. Danish BERT (monolingual)
 3. mBERT or XLM-R (multilingual)
- (Optionally: add 4th model like Flair embeddings + BiLSTM)

Ensemble Strategies:

1. Majority Voting:
 - Each model predicts BIO tags
 - Take majority vote per token
 - Resolve conflicts (e.g., B-ORG vs B-PER)
2. Weighted Voting:
 - Weight votes by model validation F1
 - Better models get more influence
3. Probability Averaging:
 - Average softmax probabilities
 - Take argmax of averaged probabilities
 - Smoother than hard voting
4. Stacking (if time):
 - Train meta-model on base model predictions
 - Logistic regression or simple NN as meta-model

Evaluation:

- Compare ensemble F1 to best single model
- Analyze when ensemble helps:
 - Does it fix errors all models make? (no)
 - Does it fix errors where models disagree? (yes)
- Error analysis: show examples

Expected Results:

- Ensemble improves by 0.5-1.5% F1 over best single model
- Gains mostly from recall (finding more entities)
- Diminishing returns beyond 3 models

Why Interesting:

- Ensemble is professor's suggestion
- Combines different model strengths
- Shows limits of single-model optimization
- Practical technique used in competitions

=====

PART 6: RECOMMENDATION AND IMPLEMENTATION TIPS

=====

6.1 Which Project Idea to Choose?

Based on professor's guidance and project constraints:

RECOMMENDED: Project Idea 1 (Cross-Lingual Transfer Learning)

Reasoning:

- Directly addresses "transfer learning" suggestion
- Uses both BiLSTM and BERT (clear baseline hierarchy)
- Produces multiple experiments (zero-shot, few-shot, full)
- Strong story: shows modern NLP techniques
- Manageable scope for course project
- Can generate nice learning curves and plots

ALTERNATIVE: Project Idea 2 (BERT Variant Comparison)

Reasoning:

- Directly addresses "systematic comparison of different BERT models"
- Clear experimental design
- Practical value (which model to use?)
- Easier to implement (no transfer complexity)
- Good for limited time

You can also COMBINE:

- Main: Transfer learning (Idea 1)
- Bonus: Compare mBERT vs XLM-R vs Danish BERT for transfer
- Gives you transfer learning + BERT comparison

6.2 Implementation Roadmap

Week 1-2: Setup and Baseline

- Load DaNE dataset
- Implement data preprocessing
- Train BiLSTM baseline
- Train BERT baseline (e.g., mBERT)
- Evaluate both, ensure code works
- Deliverable: Working BiLSTM + BERT on DaNE

Week 3-4: Core Experiments

- Load English CoNLL 2003 (if doing transfer)
- Implement transfer learning pipeline
- Run zero-shot, few-shot experiments
- OR: Train multiple BERT variants (if doing Idea 2)
- OR: Implement data augmentation (if doing Idea 3)
- Deliverable: Main experimental results

Week 5-6: Analysis and Comparison

- Error analysis
- Learning curves

- Statistical significance tests
- Per-entity-type breakdown
- Make plots and tables
- Deliverable: Complete analysis

Week 7-8: Report Writing

- Introduction and related work
- Methodology section
- Results and discussion
- Conclusion
- Make sure to cite relevant papers (BERT, DaNE, CoNLL 2003)

6.3 Key Libraries and Tools

Frameworks:

- PyTorch (recommended) or TensorFlow
- Hugging Face Transformers (essential for BERT)
- seqeval library (for NER evaluation - span-F1)

Data:

- DaNE: Available from Universal Dependencies or DaNLP
- CoNLL 2003: Available from official source or Hugging Face datasets

Models:

- Hugging Face model hub:
 - bert-base-multilingual-cased
 - xlm-roberta-base / xlm-roberta-large
 - Certainly/danish-bert-botxo
 - vesteinn/DanskBERT

Compute:

- Use ITU HPC cluster (acltr queue for GPU jobs)
- Or Google Colab with GPU (free tier might be enough for BiLSTM)
- BERT fine-tuning needs GPU (will be slow on CPU)

Evaluation:

- seqeval.metrics.classification_report
- Returns span-F1, precision, recall per entity type
- DO NOT use sklearn metrics (they do token-level, not span-level)

6.4 Common Pitfalls to Avoid

1. Wrong Evaluation Metric:

- DO NOT use token-level accuracy
- MUST use span-level F1 (seqeval library)

2. Data Leakage:

- Do NOT tune hyperparameters on test set
- Use validation set for model selection
- Test set only touched once at the very end

3. Subword Alignment:

- BERT splits words into subwords
- Only predict on first subword of each word
- Or use special alignment strategies
- Ignore ## tokens in loss calculation

4. Overfitting:

- BERT can overfit on small datasets
- Use dropout, early stopping
- Monitor validation F1, stop when it stops improving

5. BIO Violations:

- Use CRF layer to enforce valid transitions
- OR post-process predictions to fix violations
- Example fix: I-PER at start → change to B-PER

6. Imbalanced Classes:

- Don't worry too much (common in NER)
- F1 handles this better than accuracy
- Can use class weights if needed

7. Reproducibility:

- Set random seeds (Python, NumPy, PyTorch)
- Document hyperparameters
- Save model checkpoints

6.5 Expected Performance Benchmarks

DaNE (Danish NER) Benchmarks:

Baseline Models:

- Random: ~5% F1
- Majority class (all O): 0% F1
- BiLSTM: ~85-87% F1
- BiLSTM + CRF: ~87-89% F1
- BERT (mBERT/Danish BERT): ~89-92% F1

State-of-the-art:

- XLM-R large + fine-tuning: ~92-93% F1

For your project:

- Target: BiLSTM ~85%, BERT ~90%
- If you get close to these, you're doing well!
- Don't expect to beat published state-of-the-art (they have more resources)

6.6 Final Checklist Before Submission

Baselines implemented and working

- BiLSTM trained and evaluated
- BERT trained and evaluated

Advanced comparison completed

- Transfer learning experiments OR

- BERT variant comparison OR
- Data augmentation study OR
- Ensemble methods

Evaluation done correctly

- Using span-F1 (not token accuracy)
- Per-entity-type breakdown
- Statistical significance if possible

Analysis and visualization

- Learning curves
- Error analysis with examples
- Tables comparing models

Report structure

- Clear motivation (why this matters)
- Methodology explained (can reproduce)
- Results presented clearly
- Discussion of limitations
- Future work suggestions

Code and reproducibility

- Code is clean and documented
- Random seeds set
- Can re-run experiments
- README with instructions

=====

END OF NOTES

=====

Good luck with your Danish NER project!

Remember:

- Use BiLSTM or BERT as baseline (NOT logistic regression)
- Make it interesting with transfer learning, augmentation, ensembles, or BERT comparison
- Focus on span-F1 evaluation
- Show systematic evaluation and analysis

You've got this!

=====

BONUS SECTION: LANGUAGE PROFICIENCY - DO YOU NEED TO SPEAK DANISH?

=====

Short answer: ****NO**, you do NOT need to speak Danish to do this project successfully!******

7.1 Why Language Knowledge is NOT Required

1. ****The Task is Pattern Recognition, Not Understanding****

- NER models learn statistical patterns from labeled data

- The model doesn't "understand" Danish - it learns:
 - * Capitalization patterns
 - * Word position in sentences
 - * Co-occurrence patterns
 - * Subword structure
- Example: Model learns "København" appears with B-LOC label, not that it's the capital of Denmark

2. ****You Work With Pre-Labeled Data****

- DaNE dataset is already fully annotated
- Training data: sentences with BIO tags already assigned
- You don't need to label anything yourself
- You just load the data and train models

3. ****Evaluation is Automatic****

- Span-F1 metric compares predicted tags vs gold tags
- Completely automated (seqeval library)
- No human judgment needed
- You never need to read or interpret Danish sentences

4. ****Pre-trained Models Already "Know" Danish****

- BERT models pre-trained on Danish text
- They've already learned Danish linguistic patterns
- You're fine-tuning, not teaching from scratch

7.2 When Language Knowledge WOULD Help (But Isn't Necessary)

****Error Analysis:****

- Understanding why model failed on specific examples
- Example: Model tags "Danske Bank" as two separate entities
 - With Danish: You'd recognize it's a single bank name
 - Without Danish: You can still see it's a pattern error (two consecutive ORG entities)

****Qualitative Insights:****

- Explaining why Danish compounds are challenging
- Understanding cultural context of entities
- But: You can cite published papers explaining these challenges

****Data Augmentation:****

- Entity swapping requires Danish name/place lists
- Solution: Use publicly available gazetteers (Danish name lists online)
- You don't need to generate the names yourself

7.3 Practical Workarounds for Non-Danish Speakers

****For Error Analysis:****

- Use Google Translate to understand example sentences
- Focus on quantitative analysis (confusion matrices, F1 per entity type)
- Compare error patterns across models
- You can say: "Model A struggles with ORG entities (65% F1)" without understanding specific errors

****For Writing the Report:****

- Cite existing DaNE dataset papers for Danish language challenges
- Focus on technical/methodological contributions
- Your contribution is the systematic comparison, not linguistic analysis

****For Data Inspection:****

- You can visually inspect BIO tags without understanding words:

```

```
Danske B-ORG
Bank I-ORG
har O
kontor O
i O
København B-LOC
```

```

Pattern is clear even if you don't know what "har kontor i" means

****For Augmentation:****

- Use existing resources:
 - Danish person names: Statistics Denmark publishes popular names
 - Danish companies: Public company registers
 - Danish cities: Wikipedia lists
 - No language knowledge needed to swap "København" with "Aalborg"

7.4 Comparison: Danish vs English for This Project

If you chose ****English CoNLL 2003**** instead:

****Advantages:****

- You can read and understand errors easily
- More intuitive error analysis
- Easier to explain examples in report

****Disadvantages:****

- VERY well-studied benchmark (hundreds of papers)
- Hard to find novel angle
- Professor might expect higher baseline (more competition)
- Harder to make interesting contribution

****Danish NER advantages:****

- Less saturated research area
- Interesting linguistic challenges (compounds) you can cite
- Cross-lingual transfer from English is a natural experiment
- Lower baseline expectations (fewer published results)
- Living in Copenhagen - nice connection!

7.5 Real-World Analogy

Think of it like training a dog:

- You don't need to understand why "sit" means sit
- You just reward the dog when it sits after hearing "sit"
- Similarly, model doesn't understand "København"

- It just learns: capitalized word at position X often gets B-LOC label

Or like image classification:

- You can train a model to recognize Danish traffic signs
- You don't need to read Danish to know if model correctly classified a sign
- You just compare: predicted class = STOP sign, true class = STOP sign ✓

7.6 Recommendation

****Stick with Danish NER!****

Reasons:

1. You're at ITU in Copenhagen - geographic relevance
2. Less competition in research (easier to make contribution)
3. Natural cross-lingual experiments (English → Danish)
4. You don't need Danish proficiency for 95% of the work
5. For the 5% where it helps (error analysis), use Google Translate

****When to consider English instead:****

- If you want very deep qualitative error analysis
- If you're doing linguistics-focused project (you're not - this is ML)
- If you're uncomfortable with any uncertainty

But honestly: ****the technical work is identical regardless of language****

7.7 What You Actually Need to Succeed

****Essential (must have):****

- Python programming
- Understanding of neural networks (BiLSTM, BERT architecture)
- Ability to use PyTorch/Hugging Face libraries
- Understanding of evaluation metrics (span-F1)
- Systematic experimental design

****Nice to have:****

- Danish language knowledge → helpful for error analysis
- But you can compensate with:
 - Google Translate
 - Focusing on quantitative analysis
 - Citing existing papers on Danish NER challenges

****Not needed at all:****

- Being able to speak Danish fluently
- Understanding Danish grammar rules
- Knowing Danish culture

7.8 Bottom Line

****Language knowledge impact on project success: ~5-10%****

95% of the project is:

- Loading data (language-agnostic)
- Preprocessing (same code for any language)
- Training models (same architecture)
- Computing metrics (automated)
- Making plots and tables (quantitative)
- Writing methodology (technical)

5% where language helps:

- Cherry-picking good/bad examples for report
- Deep qualitative error analysis
- Understanding why specific patterns emerge

And that 5% can be handled with:

- Google Translate
- Focusing on quantitative patterns
- Citing relevant papers

****Conclusion:** Don't let language be a barrier! Danish NER is a great choice even if you don't speak Danish. ******

=====

WHY DANISH IS ACTUALLY A GOOD LANGUAGE FOR NER (RESEARCH PERSPECTIVE)

=====

Let me clarify - I should distinguish between:

1. ****Practical reasons**** (for your course project)
2. ****Linguistic/technical reasons**** (why Danish is interesting for NER research)

8.1 Why Danish is INTERESTING for NER Research

****1. Compound Words - A Genuine Challenge****

Danish forms very long compound words without spaces:

- "Sundhedsstyrelsen" (health authority) = Sundhed + s + styrelse + n
- "Udenrigsministeriet" (foreign ministry) = Uden + rig + s + ministre + t
- "Motorcenter" (motor center)

Why this matters for NER:

- Traditional word-based models fail (out-of-vocabulary problem)
- Forces you to use subword tokenization (BERT, BPE)
- Great opportunity to demonstrate WHY subword methods are important
- You can actually measure: word-level vs subword performance gap

Comparison:

- English: "Health Authority" (2 words, easy to handle)
- Danish: "Sundhedsstyrelsen" (1 word, needs splitting)

This is a ****real technical challenge**** that makes your project more interesting!

****2. Morphological Complexity****

Danish has rich morphology:

- Case markers (genitive 's)
- Definiteness suffixes (-en, -et, -ne)
- Example: "bank" vs "banken" (the bank) vs "bankens" (the bank's)

Why this matters:

- Same entity appears in multiple forms
- Tests model's ability to generalize
- More challenging than English (simpler morphology)

For NER:

- Model must learn: "Danske Bank" and "Danske Banken" are both ORG
- Morphological variation = more interesting patterns to learn

****3. Limited Capitalization Cues****

Danish capitalization:

- Sentence-initial words always capitalized (like English)
- BUT: Common nouns can be capitalized in older texts
- Nationalities/languages lowercase: "dansk", "engelsk" (unlike German)

Why this matters:

- Can't rely solely on capitalization (unlike English where proper nouns are obvious)
- Makes NER harder → more interesting to study
- Tests whether model learns contextual clues vs surface features

****4. Cross-lingual Transfer Potential****

Danish position:

- Germanic language (related to English, German, Swedish)
- But: smaller resource compared to English
- Perfect for transfer learning experiments!

Why this is academically interesting:

- Does transfer from English work despite morphological differences?
- How much Danish data needed to match English-only performance?
- Practical value: applies to ALL low-resource languages
- Novel contribution: systematic study of English→Danish transfer

Comparison:

- English→French transfer: well-studied
- English→Danish NER: less explored

****5. Low-Resource Language Characteristics****

Danish compared to English:

- Fewer pre-trained models
- Smaller datasets (DaNE ~5,600 sentences vs English CoNLL ~14,000)
- Less research attention

Why this is GOOD for your project:

- Your work actually matters! (Not just reproducing known results)

- Shows techniques applicable to other low-resource languages
- More room for novel findings
- Easier to make a contribution

8.2 Why Danish is GOOD for Your Course Project

****1. Goldilocks Zone of Difficulty****

Too easy:

- English: Everything already done, hard to contribute
- Perfect baseline models exist
- Professor expects near state-of-the-art

Too hard:

- Very low-resource languages (no datasets, no pre-trained models)
- Would spend entire project on data collection

Danish: Just right!

- Has quality dataset (DaNE)
- Has pre-trained models (Danish BERT, mBERT)
- BUT less saturated than English
- Interesting challenges (compounds) you can highlight
- Manageable scope for semester project

****2. Multiple Experiment Angles****

Danish enables interesting comparisons:

a) ****Monolingual vs Multilingual:****

- Danish BERT (monolingual) vs mBERT (multilingual)
- Which works better for Danish? (Open question!)

b) ****Cross-lingual Transfer:****

- English CoNLL → Danish DaNE
- Natural experiment (both Germanic, similar entity types)

c) ****Subword vs Word-level:****

- Show compound words NEED subword tokenization
- Demonstrate technical understanding

d) ****Low-resource Scenarios:****

- 10%, 25%, 50% of DaNE data
- Where does transfer help most?

All of these are natural with Danish, harder to justify with English!

****3. Academic Story****

Strong narrative for your report:

"We study Danish NER because:

- Danish has challenging compound words that test subword tokenization

- As a low-resource language, it benefits from transfer learning
- Findings generalize to other Nordic/Germanic languages
- Practical application: improving NER for Danish news, social media, etc."

Vs English story:

"We study English NER because... everyone else does too?"

Danish gives you a ****motivation**** beyond "it's in the course materials."

****4. Practical Relevance (for your report introduction)****

Danish NER applications:

- Danish news analysis (automatic company/person extraction)
- Danish social media monitoring
- Danish government document processing
- Nordic language technology

You're at ITU in Copenhagen → natural connection!

"As Denmark digitizes government services and processes vast amounts of Danish text..."

This reads better than:

"English is widely spoken..." (obviously, everyone knows)

8.3 When Danish Would Be BAD for NER

To be fair, Danish would be problematic if:

✗ **You need massive datasets**

- DaNE is moderate size
- English has more data if you need it

✗ **You want to use exotic architectures**

- Fewer Danish resources/tutorials
- English has more examples

✗ **Time is extremely limited**

- English has copy-paste code everywhere
- Danish needs slight adaptation

✗ **You want to beat state-of-the-art**

- Smaller research community = less pressure
- But also means fewer tricks to learn from

BUT: None of these apply to your course project!

8.4 The Real Technical Reason

****Danish is interesting because it's DIFFERENT enough from English to be challenging, but SIMILAR enough to have infrastructure.****

Sweet spot:

- Different: Compounds, morphology, smaller data → interesting challenges
- Similar: Germanic roots, same entity types, available pre-trained models → feasible

This is why linguists study it!

****Analogy:****

Learning to drive:

- Too easy: Automatic on empty highway (boring)
- Too hard: Manual in Mumbai traffic day 1 (impossible)
- Just right: Manual in suburban area (challenging but doable)

Danish NER:

- Too easy: English (everyone's done it)
- Too hard: Rare language with no resources
- Just right: Danish (interesting + feasible)

8.5 My Recommendation (Updated)

****YES, Danish is genuinely GOOD for NER research, not just acceptable.****

Reasons:

- ✓ Compound words = real technical challenge to showcase
- ✓ Low-resource = transfer learning is natural
- ✓ Less saturated = easier to contribute
- ✓ Germanic = cross-lingual experiments work
- ✓ Available resources = project is feasible
- ✓ You're in Copenhagen = nice motivation

It's not just "good enough" - it's actually an ****intelligent choice**** for a course project!

****The linguistic properties of Danish (compounds, morphology) align perfectly with modern NER techniques (subword tokenization, transfer learning) you want to demonstrate.****

That's good research design: pick a problem that showcases your methods!

For a poster presentation on your Danish Named Entity Recognition (NER) project, you should focus on the technical challenges of the Danish language, your comparative model architecture, and the systematic evaluation of modern NLP techniques.

Below are the core ideas to include, structured for a poster layout:

1. Project Motivation & The "Danish Challenge"

- **The Goal:** Developing a robust NER system to identify entities (Person, Organization, Location, Miscellaneous) specifically for Danish text.
- **Linguistic Complexity:** Highlight why Danish is a unique technical challenge compared to English:

- **Compound Words:** Danish frequently forms long compounds (e.g., *Sundhedsstyrelsen*) without spaces, which often results in Out-of-Vocabulary (OOV) issues for standard models.
- **Deep Orthography:** The complex relationship between spelling and sound requires specialized tokenization strategies.
- **Morphological Variation:** Suffixes for definiteness and genitive cases mean the same entity appears in many different forms (e.g., *bank* vs. *banken*).

2. Experimental Setup & Pipeline

- **Data:** Use the **DaNE dataset**, a standard benchmark for Danish NER consisting of approximately 5,600 annotated sentences from the news domain.
- **Preprocessing:** Implement **Subword Segmentation** (like WordPiece or BPE) to effectively handle Danish compounds.
- **Encoding:** Explain the **BIO (Beginning, Inside, Outside) scheme**, which is critical for defining multi-token entity boundaries (e.g., *Danske* [B-ORG] *Bank* [I-ORG]).

3. Model Architectures (The "Baseline Hierarchy")

Showcase a progression from sequence-aware baselines to state-of-the-art Transformers:

- **Baseline (Bi-LSTM):** A bidirectional Long Short-Term Memory network that captures context from both directions of a sentence.
 - *Key Feature:* Optional **CRF (Conditional Random Field) layer** to enforce valid BIO transitions (e.g., preventing an "I" tag from following an "O" tag).
- **Advanced (BERT/Transformers):** Fine-tuning pre-trained models like **Danish BERT (DaBERT)** or **multilingual BERT (mBERT)**.
 - *Key Feature:* Contextual embeddings and self-attention mechanisms that provide superior language understanding.

4. Advanced Comparative Studies (What Makes it "Interesting")

The professor recommends going beyond a simple comparison by including one of the following:

- **Cross-Lingual Transfer:** Training on a large English dataset (CoNLL 2003) and testing its effectiveness on Danish to see how well entity recognition transfers across languages.
- **Data Augmentation:** Measuring the impact of techniques like **Entity Swapping** or **Back-translation** on performance, especially in low-resource settings.
- **Model Variants:** Comparing monolingual Danish models against massive multilingual ones (like XLM-RoBERTa) to find the performance-efficiency "sweet spot".

5. Evaluation Metrics

- **Span-F1 Score:** Emphasize that you used **Span-F1** instead of simple accuracy. NER is a "sparse" task where most words are non-entities;

accuracy would be misleadingly high even if the model failed to find any names.

- **Per-Entity Breakdown:** Show which categories (e.g., Organization vs. Person) were the most challenging for the models.

6. Technical Implementation

- **Compute:** Mention the use of high-performance computing (HPC) resources, such as **GPU queues**, required for training complex Bi-LSTM and BERT models.
- **Frameworks:** Utilization of **PyTorch** and the **Hugging Face** Transformers library.